

## # Khoje Khatam Interview Guide

### ## Project Overview

Khoje Khatam is a React + Vite study platform for B.Tech students. It provides:

- branch-based navigation
- search and filtering by topic, year, semester, and branch
- content display for PYQs, notes, and tutorial references
- a user profile / signup UI stored locally in the frontend
- route-based deep linking for branch and subject selection

### ## Tools and Stack

- `React` for UI development
- `Vite` for tooling, bundling, and development server
- `react-router-dom` for client-side routing
- JavaScript + JSX for implementation
- `package.json` manages dependencies and scripts

### ## Key Dependencies

- `react`
- `react-dom`
- `react-router-dom`
- `vite`
- `@vitejs/plugin-react`

### ## Entry Point: `src/main.jsx`

- wraps the app with `BrowserRouter`
- defines two routes:
  - `/` renders the main `App` component
  - `/branch/:branch` renders `BranchSubjects`
- enables navigation between the home page and branch-specific pages

### ## Root App: `src/App.jsx`

#### #### Responsibilities

- manages global state for search and filters
- coordinates the main layout and content display
- handles login and user state
- reads query parameters from URL to apply filters
- renders the header, about section, branch selection, sidebar, and content panel
- shows footer with terms/privacy dialogs

#### #### State Variables

- `search` - search text query
- `selectedId` - current selected content item id
- `filterTopic` - selected topic filter
- `filterSubject` - selected subject filter
- `filterSemester` - selected semester filter

- `filterYear` - selected year filter
- `filterBranch` - selected branch filter
- `isLoggedIn`, `user` - login state and user profile
- `showProfile` - profile menu visibility
- `showTerms`, `showPrivacy` - dialog visibility
- `isHeaderMinimized` - header appearance while scrolling

#### #### Derived Data

- `branches` - unique branch options plus `All`
- `topics` - unique topics plus `All`
- `years` - unique years plus `All`

#### #### Filtering Logic

`filteredItems` is computed by checking each item against:

- search text in title, subject, content, or notes
- selected subject
- selected semester
- selected topic
- selected year
- selected branch

This is done inside `useMemo` so the filter recalculates only when dependencies change.

#### #### Selected Item

- `selectedItem` is the item matching `selectedId`
- falls back to the first filtered item if no selection exists
- shows a no-content message when nothing matches

#### #### Login Workflow

- `handleLogin(userData)` sets the user and logged-in state
- `handleLogout()` clears the user and hides the profile panel
- `ProfileMenu` receives login handlers and user data

#### #### Header Scroll Behavior

- a `useEffect` adds a scroll listener
- if the window scroll position passes 100px, the header shrinks
- this toggles `isHeaderMinimized` for responsive UI

#### #### URL Query Params

- uses `useLocation` from `react-router-dom`
- reads `branch`, `subject`, and `semester` from the URL
- applies them to filters automatically
- ensures the app can load directly into a prefiltered state

#### #### Rendered Layout

- Header with logo and `ProfileMenu`

- About section describing the app
- `BranchFilter` for branch navigation
- Sidebar with `SearchBar` and `ContentList`
- `ContentView` panel for selected content
- Footer with quick links and modals
- `TermsAndConditions` and `PrivacyPolicy` dialogs

## Profile & Signup: `src/components/ProfileMenu.jsx`

#### Purpose

- displays login/create account actions
- validates user input
- shows a student ID card after login
- manages local account state in the frontend

#### Core Features

- form state with `useState`
- input validation for name, email, branch, year, and password
- branch dropdown and year selection buttons
- local login handling without a backend API
- conditional rendering for login form vs profile card
- guest browsing hint

#### Validation Rules

- `name` must not be blank
- `email` must not be blank and must include `@`
- `branch` must be selected
- `year` must be selected
- `password` must be at least 6 characters

#### Account Creation Behavior

- on successful validation, `onLogin()` is called
- user data is stored in app state
- no actual backend API is required for the demo

## Branch Navigation: `src/components/BranchFilter.jsx`

#### Responsibilities

- renders branch buttons for each available branch
- includes an "All Branches" button
- highlights the active branch
- navigates to `/branch/:branch` using `useNavigate`

#### Behavior

- clicking a branch updates `filterBranch`
- also routes to the branch page
- branch icons are selected by branch name

## Search and Filter UI: `src/components/SearchBar.jsx`

### ### Purpose

- allows the user to search by keyword
- filters by topic
- year filter UI is present but commented out

### ### Props

- `value`, `onChange` for search text
- `topics`, `years` for filter options
- `selectedTopic`, `selectedYear` for selected values
- `onTopicChange`, `onYearChange` callbacks

## ## Branch Subjects Page: `src/pages/BranchSubjects.jsx`

### ### Purpose

- shows subject choices for the selected branch
- supports semester-based subject grouping
- navigates back to the home page with query filters

### ### Logic

- reads `branch` from route params
- collects subjects for the branch from `contentItems`
- collects semester options if available
- if semester metadata exists, the user must choose a semester first
- otherwise, subjects appear immediately

### ### Navigation

- subject buttons navigate to `/?subject=...&branch=...` or `/?subject=...&branch=...&semester=...`
- back button returns to the previous page

## ## Data Source: `src/data/pyqs.js`

- contains structured study content items
- each item includes fields like:
  - `id`
  - `branch`
  - `semester`
  - `subject`
  - `topic`
  - `year`
  - `title`
  - `content`
  - `notes`
- used for filtering and rendering across the app

## ## App Architecture

- component-based React architecture
- `App.jsx` holds global state and coordinates child components
- small reusable UI components keep code modular

- routing separates home and branch subject pages
- data-driven UI with computed options and filters
- local user session handled in-memory

## ## Interview Talking Points

### ### Why React + Vite?

- React is ideal for interactive UI and state-driven views
- Vite gives a fast local dev server and optimized builds
- `react-router-dom` enables route-based navigation

### ### Why useMemo?

- prevents expensive recalculation of filter results and option lists
- improves performance when state changes in unrelated areas

### ### Why useEffect?

- for side effects like scroll handling and URL parameter syncing
- keeps the UI in sync with browser state

### ### Component Communication

- `App` passes state and callbacks to child components
- child components notify `App` when filter values change
- `ProfileMenu` updates login state through `onLogin`
- `BranchFilter` triggers navigation and branch selection

### ### Conditional Rendering

- the app shows different UI states for logged in vs guest
- branch pages show semester buttons only when relevant
- `ContentView` is hidden when no item matches

### ### Potential Improvements

- add real API/backend for persistent login
- add user storage in localStorage or database
- make year filter visible and active
- add pagination or infinite scroll for large content lists
- improve responsive styling and mobile layout

## ## Run Instructions

1. Install dependencies:
  - `npm install`
2. Start development server:
  - `npm run dev`
3. Open the app in the browser.

## ## Conclusion

This project demonstrates:

- React component composition
- state and derived state management

- filtering and routing logic
- form validation and user experience
- data-driven UI rendering

This document is ready for interview preparation and can be used to explain architecture, feature implementation, and design decisions.